

Module 0

Installation and Setup

Learning objectives

- Set up development environment and verify installation

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["System Requirements and Prer"]

T2["Verilator Installation"]

T1 --> T2

T3["UVM Library Installation"]

T2 --> T3

Set up development environment and verify installation

Overview

- This module covers the complete setup of your verification environment, including all required tool...

Syllabus topics

1. System Requirements and Prerequisites (1/3)

- Operating System Support
- Linux (Ubuntu/Debian, CentOS/RHEL, Fedora)
- macOS (Intel and Apple Silicon)
- Windows (WSL2 recommended)
- Hardware Requirements
- Minimum 8GB RAM (16GB+ recommended)

1. System Requirements and Prerequisites (2/3)

- 10GB free disk space
- Multi-core processor recommended
- Software Prerequisites
 - C/C++ compiler (GCC 7+, Clang 10+, or MSVC)
 - Make or Ninja build system
 - Git

1. System Requirements and Prerequisites (3/3)

- Perl (for UVM library)

2. Verilator Installation (1/6)

- What is Verilator?
- Open-source Verilog/SystemVerilog simulator
- Fast compilation and simulation
- Limited SystemVerilog support (see limitations section)
- Automated Installation (Recommended)
- Using the installation script:

2. Verilator Installation (2/6)

- The script automatically:
- Checks for existing installations
- Installs system dependencies (build tools, libraries)
- Sets up git submodule in `tools/verilator/`
- Builds and installs Verilator
- Verifies the installation

2. Verilator Installation (3/6)

- Manual Installation Methods
- Linux Installation
- Ubuntu/Debian: ``sudo apt-get install verilator``
- CentOS/RHEL: ``sudo yum install verilator`` or ``sudo dnf install verilator``
- Fedora: ``sudo dnf install verilator``
- Building from source (latest features)

2. Verilator Installation (4/6)

- macOS Installation
- Homebrew installation: ``brew install verilator``
- MacPorts installation (alternative)
- Building from source
- Windows/WSL2 Installation
- Installing in WSL2 Ubuntu: ``sudo apt-get install verilator``

2. Verilator Installation (5/6)

- Building from source in WSL2
- Verifying installation
- Uninstallation
- Verification Steps
- Check Verilator version: ``verilator --version``
- Run simple Verilator test

2. Verilator Installation (6/6)

- Verify compilation works

3. UVM Library Installation (1/5)

- What is UVM?
- Universal Verification Methodology
- IEEE 1800.2-2017 standard
- Industry-standard verification framework
- Automated Installation (Recommended)
- Using the installation script:

3. UVM Library Installation (2/5)

- The script automatically:
- Downloads Accellera UVM 2017-1.0 tarball to
 ``tools/uvm-2017/``
- Extracts and sets up UVM library
- Creates environment setup script
 (``scripts/setup_uvm_env.sh``)
- Configures UVM library paths
- Manual Installation Methods

3. UVM Library Installation (3/5)

- From Accellera Tarball:
- From System Package (if available):
- Some Linux distributions provide UVM packages
- Check your distribution's package manager
- Uninstallation
- Environment Variables

3. UVM Library Installation (4/5)

- ``UVM_HOME``: Path to UVM library
- ``UVM_VERBOSITY``: Default verbosity level
- Include paths for compilation
- Verification Steps
- Check UVM library exists: ``ls $UVM_HOME/uvm_pkg.sv``
(after sourcing ``scripts/setup_uvm_env.sh``)
- Compile simple UVM test

3. UVM Library Installation (5/5)

- Verify UVM classes available

4. Commercial Simulator Setup (Optional) (1/2)

- Synopsys VCS
- Installation from Synopsys
- License setup
- UVM library integration
- Mentor QuestaSim/ModelSim
- Installation from Siemens

4. Commercial Simulator Setup (Optional) (2/2)

- License setup
- UVM library integration
- Cadence Xcelium
- Installation from Cadence
- License setup
- UVM library integration

5. Build System Configuration (1/3)

- Makefile Setup
- Basic Makefile for Verilator
- Compilation flags
- Include paths
- Test execution
- Verilator Compilation

5. Build System Configuration (2/3)

- SystemVerilog flags: ``-sv``, ``--timing``
- Include paths: ``+incdir+$UVM_HOME`` (UVM_HOME already points to the src directory)
- Optimization flags
- Coverage options
- Commercial Simulator Setup
- Compilation scripts

5. Build System Configuration (3/3)

- Simulation scripts
- Waveform generation

6. IDE Setup and Configuration (1/3)

- Recommended IDEs
- VS Code with SystemVerilog extension
- Verible (SystemVerilog linter/formatter)
- Sigasi Studio (commercial)
- Vim/Neovim with LSP
- VS Code Configuration

6. IDE Setup and Configuration (2/3)

- SystemVerilog extension setup
- Verilator integration
- Debugging configuration
- Task runner setup for simulations
- Extension recommendations
- Editor Configuration

6. IDE Setup and Configuration (3/3)

- SystemVerilog formatting (Verible)
- Linting (Verilator, Verible)
- Code formatting on save
- Syntax highlighting

7. Project Structure Setup (1/4)

- Directory Structure
- Source code organization
- Test directory structure
- DUT (Design Under Test) organization
- Configuration files
- `tools/` directory for tools (Verilator as git submodule, UVM as tarball)

7. Project Structure Setup (2/4)

- Git Submodules Management
- Verilator is managed as a git submodule in
 ``tools/verilator/``
- UVM is downloaded as a tarball to ``tools/uvm-2017/``
 (not a git submodule)
- Initialize submodules:
 ``./scripts/init_submodules.sh`` or ``git submodule
 update --init --recursive``
- Update submodules: ``./scripts/update_submodules.sh``
 or ``git submodule update --remote``
- Add new submodule: ``./scripts/add_submodule.sh
 <repo_url> <path>``

7. Project Structure Setup (3/4)

- Remove submodule: ``./scripts/remove_submodule.sh <path>``
- Makefile/Configuration
- Simple Makefile for running tests
- Verilator configuration
- Environment variable management
- Version Control

7. Project Structure Setup (4/4)

- Git initialization
- .gitignore for SystemVerilog and simulation (include build artifacts, waveforms)
- Initial commit structure
- Git submodules in `.gitmodules` file (only Verilator, not UVM)

8. First "Hello World" Verification Test (1/3)

- Prerequisites
- Ensure all tools are installed:
 ``./scripts/installation.sh``
- Verify tools are accessible: ``verilator --version``
 and ``ls $UVM_HOME/uvm_pkg.sv`` (after sourcing ``s...`
- Simple DUT Creation
- Basic Verilog module (e.g., AND gate)
- Simple testbench structure

8. First "Hello World" Verification Test (2/3)

- SystemVerilog Test
- Basic SystemVerilog testbench structure
- Clock generation
- Signal driving and reading
- Running the test
- UVM Test

8. First "Hello World" Verification Test (3/3)

- First UVM test class
- Basic UVM phases
- Running UVM test
- Understanding output

9. Verilator Limitations and Workarounds (1/3)

- Known Limitations
 - Limited SystemVerilog support
 - Some UVM classes may not work
 - Complex TLM connections may fail
 - Full assertion support limited
- Workarounds

9. Verilator Limitations and Workarounds (2/3)

- Use simplified patterns where possible
- Document commercial alternatives
- Focus on concepts over tool features
- Commercial Simulator Alternatives
- When Verilator doesn't support a feature
- How to adapt examples for commercial tools

9. Verilator Limitations and Workarounds (3/3)

- Compatibility notes

10. Troubleshooting Common Issues (1/4)

- Verilator Issues
- Compilation errors
- Missing dependencies
- Version compatibility
- Path issues
- SystemVerilog syntax errors

10. Troubleshooting Common Issues (2/4)

- UVM Library Issues
- Import errors
- Path configuration
- Version compatibility
- Compilation errors
- Build System Issues

10. Troubleshooting Common Issues (3/4)

- Makefile errors
- Include path problems
- Compilation flags
- IDE Issues
- SystemVerilog syntax highlighting
- Import resolution problems

10. Troubleshooting Common Issues (4/4)

- Debugging not working

11. Verification Checklist (1/3)

- C/C++ compiler installed and working
- Verilator installed and verified
- Using script: ``./scripts/install_verilator.sh --from-submodule``
- Verify: ``verilator --version``
- UVM library installed and verified
- Using script: ``./scripts/installation.sh`` (UVM is installed automatically)

11. Verification Checklist (2/3)

- Verify: ``ls $UVM_HOME/uvm_pkg.sv`` (after sourcing ``scripts/setup_uvm_env.sh``)
- All tools verified together: Run ``./scripts/installation.sh`` and verify with ``verilator --version`` ...
- IDE configured and working
- First test runs successfully
- Can create and run simple testbench
- Understand basic project structure

11. Verification Checklist (3/3)

- Know how to get help when stuck
- Understand Verilator limitations

Hands-on examples

Module 0 self-check

```
$ verilator --version
```

Module 0 self-check
(run ./scripts/module0.sh when ready)

Demo: Verify toolchain

```
$ verilator --version && source scripts/setup_uvm_env.sh 2>/dev/null  
|| true && test -f  
"${UVM_HOME:-tools/uvm-2017/1800.2-2017-1.0/src}/uvm_pkg.sv" &&
```

```
Terminal: ex01_install  
UVM library OK  
GNU Make 4.3  
/bin/sh: 1: verilator: not found
```


Demo: Project layout

```
$ ls -la docs/MODULE0.md module1/README.md scripts/installation.sh
```

(screenshot pending: assets/screenshots/ex02_project.png)

Demo: Optional full install

```
$ ./scripts/installation.sh --help | head -20
```

Terminal: ex03_install-full

```
[0:34m[2026-05-28 20:35:38] Starting Verilator with UVM 2017 installation...[0m
[0:34m[2026-05-28 20:35:38] Based on: https://antmicro.com/blog/2025/10/support-for-upstream-uv
Usage: ./scripts/installation.sh [OPTIONS]
```

Installs Verilator from source and sets up Accellera UVM 2017-1.0

Options:

```
--local      Install Verilator to project-local directory (no sudo required)
--prefix PREFIX  Installation prefix for Verilator (default: /usr/local)
--skip-uvm     Skip UVM download and setup
--venv DIR     Use virtual environment at DIR (default: .venv)
--no-venv     Skip Python virtual environment setup
--force       Force reinstall even if already installed
--help, -h    Show this help message
```

Examples:

```
./scripts/installation.sh      # Install Verilator and UVM 2017 to system (with ve
./scripts/installation.sh --local  # Install to project-local directory
./scripts/installation.sh --skip-uvm  # Install only Verilator
./scripts/installation.sh --no-venv  # Skip Python virtual environment setup
```

Demo: First smoke test

```
$ ./scripts/module1.sh --help | head -15
```

Terminal: ex04_hello

Usage: ./scripts/module1.sh [OPTIONS]

Module 1: SystemVerilog and Verification Basics
This script runs examples and tests for Module 1.

OPTIONS:

SystemVerilog Examples:

--sv-basics	Run SystemVerilog class basics examples
--interfaces	Run interfaces and modports examples
--packages	Run packages and namespaces examples
--data-structures	Run data structures examples
--error-handling	Run error handling and logging examples
--all-sv	Run all SystemVerilog examples (default)
--skip-sv	Skip all SystemVerilog examples

Practice & assessment

What you should know (1/2)

- Install and configure all required tools
- Set up Verilator
- Install and verify UVM library
- Configure your IDE for verification work
- Create a basic project structure
- Run a simple verification test

What you should know (2/2)

- Understand Verilator limitations
- Troubleshoot common installation issues

Exercises

- Installation Verification
- Environment Setup
- First Test
- IDE Configuration
- Project Structure

Assessment checklist

- Can install all required tools independently
- Can set up Verilator
- Can verify Verilator installation
- Can verify UVM library installation
- Can configure IDE for verification work
- Can create and run a simple test

Summary & next steps

- Set up development environment and verify installation
- Complete module0/CHECKLIST.md
- Review module0/EXAMPLES.md and run each lab
- Next: Next module in course